

Calculating Suspense, Surprise, and Shock

Travis Richardson

Table of Contents

Introduction

Step 1: Calculating Score Lines

- (a) Importing necessary libraries and defining functions
- (b) Calculating the Poisson distribution grid
- (c) Generating lambda values and precomputing probabilities
- (d) Reading match data and mapping columns
- (e) Minimization process for fitting scorelines
- (f) Extracting and saving predicted scoring rates

Step 2: Empirical Goal Distribution

Step 3: Calculating Live Betting Odds

- (a) Importing libraries, setting seed, and loading data
- (b) Extracting variables and separating scores
- (c) Simulating goal occurrences with red card adjustments
- (d) Calculating standard win probabilities
- (e) Calculating no-goal-no-red win probabilities
- (f) Processing all games and saving output

Step 4: Calculating Suspense

- (a) Importing libraries, setting seed, and loading data
- (b) Simulations and variable extraction
- (c) Calculating hypothetical win probabilities: home goal scenario
- (d) Calculating hypothetical win probabilities: away goal scenario
- (e) Forward-looking shift and suspense formula
- (f) Processing all games and saving output

Step 5: Calculating Surprise and Shock

- (a) Importing libraries and loading data
- (b) Calculating Surprise
- (c) Calculating Shock
- (d) Replacing Surprise with Shock in Minute 1
- (e) Saving output

Conclusion

Introduction

This document outlines the full procedure for generating live betting odds, a suspense measure, and surprise and shock values for soccer matches. The process relies on simulating minute-by-minute match outcomes using Poisson-distributed scoring rates, empirical goal distributions, and pre-match bookmaker odds.

Before the steps described here can be executed, several data preparation procedures must be completed. These include: (1) cleaning raw bookmaker odds by removing the overround to produce true implied probabilities; (2) extracting minute-level goal and red card events from raw match event data and transforming them into a structured format; and (3) combining per-season files into a single all-seasons dataset. Additionally, the minute-by-minute input file used in Step 3 is constructed by merging the predicted scoring rates from Step 1, the empirical goal distribution from Step 2, and the cleaned goal and red card data into a single dataset expanded to 92 rows per match. Full preprocessing scripts for all of these steps are available in the accompanying GitHub repository.

The steps described in this document are numbered 1 through 5, with lettered subsections providing additional detail. Each step builds directly on the output of the previous one, forming a complete pipeline from raw odds data to final suspense, surprise, and shock values.

Step 1: Calculating Score Lines

The goal of this step is to find, for each match, the pair of home and away scoring rates (lambda values) that best fits the observed bookmaker over/under odds. This is achieved by precomputing Poisson-based scoreline probabilities for all possible lambda combinations and then selecting the combination that minimizes the sum of squared differences between the model's predictions and the observed odds.

(a) Importing necessary libraries and defining helper functions

```
1 import pandas as pd
2 import numpy as np
3 from numpy import arange
4 from scipy.stats import poisson
5 import os
6
7 os.chdir(r"path/to/your/input/directory")
```

Explanation: This step imports the essential libraries needed for the calculations. No data processing occurs here.

- **pandas:** Used for reading, manipulating, and saving tabular data.
- **numpy:** Used for numerical operations and array manipulation.
- **poisson from scipy.stats:** Used to construct the Poisson probability mass function for goal scoring.
- **os:** Used to manage file paths and directories.

(b) Calculating the Poisson distribution grid

```
1 def score_lines_per_match(lam):
2     """
3     Given lam = [lamA, lamB], return a 10x10 matrix with probabilities
4     for each (i,j) scoreline using independent Poisson distributions.
5     """
6     goals = poisson(lam)
7     x = np.zeros([10, 10])
8     for i in range(0, x.shape[0]):
9         for j in range(0, x.shape[1]):
10             x[i, j] = goals.pmf([i, j])[0] * goals.pmf([i, j])[1]
11     return x
```

Explanation: This function constructs a 10×10 probability matrix for all probable scorelines (i, j) where i is the number of home goals and j is the number of away goals. Each cell $x[i, j]$ is computed as the product of two independent Poisson probability mass functions, one for the home team scoring i goals and one for the away team scoring j goals. The two teams are assumed to score independently of one another. This matrix forms the foundation for deriving all subsequent match outcome probabilities (win, draw, loss, over/under).

(c) Generating lambda values and precomputing all probabilities

```
1 def float_range(A, L=None, D=None):
2     """Float-based range generator."""
3     if L == None:
4         L = A + 0.0
5         A = 0.0
6     if D == None:
7         D = 1.0
8     while True:
9         if D > 0 and A >= L:
10             break
```

```

11         elif D < 0 and A <= L:
12             break
13         yield ("%g" % A)
14         A = A + D
15
16     lamdas_a = []
17     lamdas_b = []
18     for i in float_range(0, 6, 0.1):
19         lamdas_a.append(float(i))
20         lamdas_b.append(float(i))
21
22     mega = []
23     for x in lamdas_a:
24         for y in lamdas_b:
25             mega.append([x, y])
26
27     keep_stats = {}
28     for lam_pair in mega:
29         x = score_lines_per_match(lam_pair)
30         dff = pd.DataFrame(x)
31
32         totalAway = sum([x[i][j] for i in range(1, x.shape[0])
33                          for j in range(i)])
34         totalHome = sum([x[i][j] for i in range(x.shape[0])
35                          for j in range(i+1, x.shape[1])])
36         total_draw = dff.values.sum() - totalAway - totalHome
37
38         under_5 = x[0][0]
39         over_5 = dff.values.sum() - under_5
40         under_15 = under_5 + x[1][0] + x[0][1]
41         over_15 = dff.values.sum() - under_15
42         under_25 = under_15 + x[2][0] + x[0][2] + x[1][1]
43         over_25 = dff.values.sum() - under_25
44         under_35 = under_25 + x[3][0] + x[0][3] + x[2][1] + x[1][2]
45         over_35 = dff.values.sum() - under_35
46         under_45 = under_35 + x[4][0] + x[3][1] + x[2][2] + x[1][3] + x[0][4]
47         over_45 = dff.values.sum() - under_45
48         under_55 = under_45 + x[5][0] + x[4][1] + x[3][2] + x[2][3] \
49             + x[1][4] + x[0][5]
50         over_55 = dff.values.sum() - under_55
51
52         keep_stats[f'{lam_pair[0]:.1f}_{lam_pair[1]:.1f}'] = [
53             under_5, under_15, under_25, under_35, under_45, under_55,
54             over_5, over_15, over_25, over_35, over_45, over_55,
55             totalHome, totalAway, total_draw
56         ]

```

Explanation: This section generates all possible combinations of home and away scoring rates from 0.0 to 5.9 in increments of 0.1, producing 3,600 lambda pairs. For each pair, the scoreline probability matrix is computed and then aggregated into outcome probabilities:

- **Home win, away win, draw:** Derived by summing the appropriate cells of the scoreline matrix. A home win occurs when $i > j$, an away win when $j > i$, and a draw when $i = j$.
- **Over/Under thresholds:** Computed cumulatively for 0.5 through 5.5 total goals by summing the relevant scoreline cells.
- All results are stored in the `keep_stats` dictionary, keyed by a string of the form "1.2.0.8", representing the home and away lambda values. This precomputation avoids redundant calculations during the fitting step.

(d) Reading match data and mapping columns

```

1 df = pd.read_excel('input_odds_data.xlsx', engine='openpyxl')
2

```

```

3 column_map = {
4     'cav_un05': 0, 'cav_un15': 1, 'cav_un25': 2,
5     'cav_un35': 3, 'cav_un45': 4, 'cav_un55': 5,
6     'cav_ov05': 6, 'cav_ov15': 7, 'cav_ov25': 8,
7     'cav_ov35': 9, 'cav_ov45': 10, 'cav_ov55': 11,
8     'clean_h60': 12, 'clean_a60': 13, 'clean_d60': 14
9 }
10
11 lists_dict = {}
12 for col in column_map.keys():
13     if col in df.columns:
14         lists_dict[col] = df[col].astype(float).to_list()
15     else:
16         lists_dict[col] = None

```

Explanation: The cleaned bookmaker odds (produced in the preprocessing step) are loaded here. The `column_map` dictionary maps each odds column in the DataFrame to its corresponding index in the `keep_stats` list. This design is intentionally flexible: if a particular over/under line is missing for a given league or season, the script simply skips it rather than raising an error. This allows the same script to be applied across leagues with different data availability. The `lists_dict` stores each column as a Python list for fast row-by-row access during the fitting loop.

(e) Minimization process for fitting scorelines

```

1 lamdaz = []
2 paul_bearer = []
3
4 num_rows = len(df)
5 for row_idx in range(num_rows):
6     min_track = []
7     lamda_track = {}
8
9     for lam_key, stats_list in keep_stats.items():
10         sse = 0.0
11         for col, stat_idx in column_map.items():
12             if lists_dict[col] is not None:
13                 value = lists_dict[col][row_idx]
14                 if not pd.isnull(value):
15                     diff = value - stats_list[stat_idx]
16                     sse += diff**2
17
18         min_track.append(sse)
19         lamda_track[sse] = lam_key
20
21 min_val = min(min_track)
22 paul_bearer.append(min_val)
23 lamdaz.append(lamda_track[min_val])

```

Explanation: For each match, this loop searches across all 3,600 precomputed lambda pairs and selects the one that minimizes the sum of squared errors (SSE) between the model's predicted probabilities and the observed cleaned bookmaker odds. This is a grid search approach; rather than using a numerical optimizer, it exhaustively evaluates every candidate and picks the best fit. Only columns that are present and non-missing contribute to the SSE, ensuring robustness across datasets with varying availability of over/under lines. The best-fitting lambda key (e.g., "1.4_0.9") is stored for each match.

(f) Extracting and saving predicted scoring rates

```

1 home_log = []
2 away_log = []
3
4 for lam_str in lamdaz:
5     away_score_str, home_score_str = lam_str.split("_")

```

```

6     away_log.append(float(away_score_str))
7     home_log.append(float(home_score_str))
8
9 df['h_simu'] = home_log
10 df['a_simu'] = away_log
11
12 df.to_csv('predicted_goals.csv', index=False)
13 print("Done!")

```

Explanation: The best-fitting lambda string for each match is split into its home and away components and converted to float values. These represent the expected number of goals per match for each team — not per minute, but for the full game. They are appended to the DataFrame as `h_simu` (home predicted goals) and `a_simu` (away predicted goals) and saved to a CSV file for use in subsequent steps. Note that in the lambda string format "`away_home`", the first value corresponds to the away team and the second to the home team.

Conclusion of Step 1

Step 1 establishes the per-match scoring rates that underpin all subsequent calculations. By fitting Poisson-based scoreline probabilities to cleaned bookmaker odds via a grid search, we obtain a home and away expected goals value for each match. These values are not raw observed goals but model-estimated scoring rates that best explain the bookmaker's implied probabilities across all over/under markets. These rates are then distributed across the 92 minutes of each match using the empirical goal distribution in the following steps.

Step 2: Empirical Goal Distribution

Before proceeding, it is essential to construct an empirical minute-by-minute goal distribution if one has not already been prepared. This requires collecting goal-scoring data across a large number of matches and seasons, including the exact minute in which each goal was scored. From this data, the proportion of all goals scored in each minute (1 through 92) is calculated, yielding a probability distribution over the 92 minutes of a match.

Why 92 minutes? Each match is represented as exactly 92 minutes rather than 90. This is because stoppage time is condensed into fixed boundary minutes rather than treated as additional minutes beyond 90. Specifically, the match is structured as follows:

- **First half (minutes 1–46):** The first half covers minutes 1 through 45 as normal. Any goals or events occurring in first-half stoppage time (45+1, 45+2, 45+3, etc.) are all condensed into minute 46. So minute 46 represents the entirety of first-half injury time.
- **Second half (minutes 47–92):** The second half begins at minute 47 rather than minute 46. This is because all second-half minutes are shifted forward by one to accommodate the first-half stoppage time minute. Regular second-half play covers minutes 47 through 91 (corresponding to match minutes 46 through 90). Any goals or events occurring in second-half stoppage time (90+1, 90+2, 90+3, etc.) are all condensed into minute 92. So minute 92 represents the entirety of second-half injury time.

This structure means that minute 46 and minute 92 will naturally have higher goal frequencies than surrounding minutes in the empirical distribution, since they each absorb all stoppage time events from their respective halves. This is an accurate reflection of real match dynamics, where a disproportionate number of goals are scored in injury time.

This empirical distribution is later used to scale the per-match scoring rates from Step 1 into per-minute scoring probabilities. For example, if a team is expected to score 1.5 goals in a match and historically 2% of all goals are scored in minute 30, then the per-minute scoring rate for that team in minute 30 is $1.5 \times 0.02 = 0.03$. These per-minute rates are what drive the binomial simulations in Steps 3 and 4.

The empirical distribution should be computed separately for each league, as goal timing patterns can differ across competitions. The output is a simple two-column file containing the minute (1–92) and the corresponding proportion of goals scored in that minute.

Introduction to Steps 3 and 4

Steps 3 and 4 are closely related and share much of the same simulation infrastructure. Both steps simulate the remaining minutes of a match from each minute forward, using the per-minute scoring rates derived from Steps 1 and 2, and adjust for any red cards that have occurred. The key difference between the two steps is what is done with the simulation output.

Step 3 uses the simulations to calculate the live betting odds at each minute; that is, the probability of a home win, draw, or away win given the current score and remaining time. These odds serve as the baseline for the suspense measure. A second set of odds is also calculated under a “no goal, no red card” assumption, which is used as the contamination-free baseline for the suspense calculation in Step 4.

Step 4 introduces hypothetical “what if” scenarios, asking how the match outcome probabilities would change if either the home or away team were to score in the next minute. The suspense measure is then derived from the magnitude of these hypothetical changes, weighted by the probability that a goal actually occurs in that next minute.

Step 3: Calculating Live Betting Odds

This step simulates the remaining match from every minute forward, producing minute-by-minute win, draw, and loss probabilities. Two sets of odds are produced: standard live odds that reflect the actual match state, and a contamination-free version that removes the influence of goals or red cards that occurred in the current minute. Both sets are saved to the output file for use in Step 4.

(a) Importing libraries, setting the random seed, and loading data

```
1 import pandas as pd
2 import numpy as np
3 import os
4
5 np.random.seed(482931)
6
7 INPUT_PATH = 'minute_by_minute_input.csv'
8 OUTPUT_PATH = 'live_betting_odds_output.csv'
9
10 dataset = pd.read_csv(INPUT_PATH)
```

Explanation: The random seed is set at the top of the script using `np.random.seed(482931)` to ensure full reproducibility. Because the simulation relies on random binomial draws, fixing the seed guarantees that anyone running the script with the same input data will obtain identical results — we do this as a requirement for replication. The input file is the minute-by-minute dataset constructed during preprocessing, containing one row per minute per match with columns for the current score, red card counts, per-minute scoring rates, and pre-match probabilities.

(b) Extracting variables and separating scores

```
1 def score_separator(score):
2     homescore_now = []
3     awayscore_now = []
4     for i in score:
5         home, away = map(int, i.split(','))
6         homescore_now.append(home)
7         awayscore_now.append(away)
8     return homescore_now, awayscore_now
9
10 def all_variables(dataset):
11     score = dataset['score'].dropna()
12     red = dataset['red'].dropna().tolist()
13     homered, awayred = score_separator(red)
14     homescored, awayscored = score_separator(score)
15     homescoring = dataset['homescoring'].dropna().values
16     awayscoring = dataset['awayscoring'].dropna().values
17     home_win = dataset['prehome'].dropna().values
18     all_draw = dataset['predraw'].dropna().values
19     away_win = dataset['preaway'].dropna().values
20     return [
21         homered, awayred,
22         homescored, awayscored,
23         homescoring, awayscoring,
24         home_win, all_draw, away_win
25     ]
```

Explanation: The `score_separator()` function parses the cumulative score strings (e.g., "2,1") and red card strings into separate integer lists for home and away values. This is applied to both the score column (tracking goals) and the red column (tracking red cards accumulated by each team up to that minute).

The `all_variables()` function collects all key variables from the dataset into a structured list. In Step 3, the baseline odds `home_win`, `all_draw`, and `away_win` are taken from the pre-match closing odds (`prehome`, `predraw`, `preaway`), which represent the bookmaker's assessment of match outcome before kick-off.

(c) Simulating goal occurrences with red card adjustments

```
1 def simulations(homered, awayred, homescoring, awayscoring):
2     homered = np.array(homered, dtype=int)
3     awayred = np.array(awayred, dtype=int)
4     homescoring = np.array(homescoring)
5     awayscoring = np.array(awayscoring)
6     num_minutes = len(homescoring)
7
8     adjusted_homescoring = homescoring.copy()
9     adjusted_awayscoring = awayscoring.copy()
10
11     mask_home_more_red = homered > awayred
12     adjusted_homescoring[mask_home_more_red] *= (2 / 3)
13     adjusted_awayscoring[mask_home_more_red] *= 1.2
14
15     mask_away_more_red = homered < awayred
16     adjusted_homescoring[mask_away_more_red] *= 1.2
17     adjusted_awayscoring[mask_away_more_red] *= (2 / 3)
18
19     num_samples = 100000
20     home_binomials = np.random.binomial(
21         n=1, p=adjusted_homescoring[:, None],
22         size=(num_minutes, num_samples)
23     )
24     away_binomials = np.random.binomial(
25         n=1, p=adjusted_awayscoring[:, None],
26         size=(num_minutes, num_samples)
27     )
28
29     return home_binomials, away_binomials
```

Explanation: This function generates the core simulation output: two matrices of shape (`num_minutes` × `num_samples`), where each cell is a binary indicator of whether a goal was scored in that minute for that simulated future. The entire game’s binomials are generated at once using vectorized numpy operations, which is substantially faster than the minute-by-minute loop used in earlier versions.

Red card adjustments are applied to the per-minute scoring rates *before* the binomials are generated, using vectorized boolean masking:

- If the home team has more red cards than the away team at a given minute, the home team’s scoring rate is multiplied by $\frac{2}{3}$ and the away team’s scoring rate is multiplied by 1.2 for all remaining minutes from that point.
- The reverse applies if the away team has more red cards.
- If both teams have equal red cards (including zero), no adjustment is made.

Crucially, these adjustments are permanently baked into the generated binomials. This means that downstream functions using these binomials do not need to re-apply any red card logic — doing so would result in double-penalization.

(d) Calculating standard win probabilities

```
1 def calculate_chances(home_binomials, away_binomials,
2                       homescored, awayscored):
3     num_minutes, num_samples = home_binomials.shape
4     homewin_chance = np.zeros(num_minutes)
5     draw_chance = np.zeros(num_minutes)
6     awaywin_chance = np.zeros(num_minutes)
7
8     for i in range(num_minutes):
9         home_goals = home_binomials[i:, :].sum(axis=0) + homescored[i]
10        away_goals = away_binomials[i:, :].sum(axis=0) + awayscored[i]
```

```

12     differences = home_goals - away_goals
13     home_wins = np.sum(differences > 0)
14     draws = np.sum(differences == 0)
15     away_wins = np.sum(differences < 0)
16
17     total = home_wins + draws + away_wins
18     homewin_chance[i] = home_wins / total
19     draw_chance[i] = draws / total
20     awaywin_chance[i] = away_wins / total
21
22     return homewin_chance, draw_chance, awaywin_chance

```

Explanation: For each minute i , this function simulates the total goals scored from minute i onwards by summing the binomial draws for all remaining minutes (`home_binomials[i:, :].sum(axis=0)`). The current actual score is then added to reflect the true match state. The difference in total goals across all 100,000 simulated futures determines the distribution of outcomes, from which home win, draw, and away win probabilities are calculated. These probabilities represent the live betting odds at each minute given the actual score and time remaining.

(e) Calculating no-goal-no-red win probabilities

```

1 def calculate_chances_nogoalnoered(home_binomials, away_binomials,
2                                     homescored, awayscored,
3                                     homered, awayred):
4     """
5     Identical to calculate_chances except that for any minute where a goal
6     or red card occurred, we substitute the PREVIOUS minute's score and
7     red card state, pretending that minute's event did not happen.
8     Red card adjustment is already baked into the binomials from simulations()
9     so we do not re-adjust here.
10    """
11    num_minutes, num_samples = home_binomials.shape
12    homewin_chance = np.zeros(num_minutes)
13    draw_chance = np.zeros(num_minutes)
14    awaywin_chance = np.zeros(num_minutes)
15
16    for i in range(num_minutes):
17        if i == 0:
18            home_goals_now = homescored[i]
19            away_goals_now = awayscored[i]
20        else:
21            goal_this_minute = (homescored[i] != homescored[i-1] or
22                                awayscored[i] != awayscored[i-1])
23            red_this_minute = (homered[i] != homered[i-1] or
24                               awayred[i] != awayred[i-1])
25
26            if goal_this_minute or red_this_minute:
27                home_goals_now = homescored[i - 1]
28                away_goals_now = awayscored[i - 1]
29            else:
30                home_goals_now = homescored[i]
31                away_goals_now = awayscored[i]
32
33    home_goals = home_binomials[i:, :].sum(axis=0) + home_goals_now
34    away_goals = away_binomials[i:, :].sum(axis=0) + away_goals_now
35
36    differences = home_goals - away_goals
37    home_wins = np.sum(differences > 0)
38    draws = np.sum(differences == 0)
39    away_wins = np.sum(differences < 0)
40
41    total = home_wins + draws + away_wins
42    homewin_chance[i] = home_wins / total
43    draw_chance[i] = draws / total
44    awaywin_chance[i] = away_wins / total

```

```

45     return homewin_chance, draw_chance, awaywin_chance
46

```

Explanation: This function calculates what the live odds would have been at each minute if no goal or red card had occurred in that minute. Rather than reflecting the actual match state, it uses the previous minute's cumulative score and red card count as the baseline whenever a goal or red card is detected — effectively pretending that the event did not happen. Minute 1 is treated as a special case and always uses the actual state since there is no previous minute to step back to.

This produces a counterfactual set of live odds that isolates the underlying match dynamics from the influence of discrete match events. These odds can be used for alternative analyses or robustness checks where it is important to separate the effect of goals and red cards from the baseline match state — for example, examining how suspense or surprise measures would behave in the absence of such events. Importantly, the binomials are used directly without re-applying red card adjustments, since those adjustments were already incorporated when the binomials were generated in `simulations()`.

(f) Processing all games and saving output

```

1  def game_betting_odds(game_data):
2      (homered, awayred, homescored, awayscored,
3       homescoring, awayscoring,
4       home_win, all_draw, away_win) = game_data
5
6      home_binomials, away_binomials = simulations(
7          homered, awayred, homescoring, awayscoring
8      )
9
10     homewin_chance, draw_chance, awaywin_chance = calculate_chances(
11         home_binomials, away_binomials, homescored, awayscored
12     )
13
14     homewin_nogoalnored, draw_nogoalnored, awaywin_nogoalnored = \
15         calculate_chances_nogoalnored(
16             home_binomials, away_binomials,
17             homescored, awayscored,
18             homered, awayred
19         )
20
21     return (homewin_chance, draw_chance, awaywin_chance,
22            homewin_nogoalnored, draw_nogoalnored, awaywin_nogoalnored)
23
24 def process_games(dataset):
25     num_minutes = 92
26     num_games = len(dataset) // num_minutes
27     results = []
28
29     for game_idx in range(num_games):
30         start = game_idx * num_minutes
31         end = start + num_minutes
32         game_data = [var[start:end] for var in all_variables(dataset)]
33
34         (homewin_chance, draw_chance, awaywin_chance,
35          homewin_nogoalnored, draw_nogoalnored,
36          awaywin_nogoalnored) = game_betting_odds(game_data)
37
38         game_results = pd.DataFrame({
39             'homewin_chance': homewin_chance,
40             'draw_chance': draw_chance,
41             'awaywin_chance': awaywin_chance,
42             'homewin_nogoalnored': homewin_nogoalnored,
43             'draw_nogoalnored': draw_nogoalnored,
44             'awaywin_nogoalnored': awaywin_nogoalnored,
45         })
46         results.append(game_results)
47         print(f"Game_{game_idx+1} done")

```

```

48     return pd.concat(results, ignore_index=True)
49
50
51 all_results = process_games(dataset)
52
53 df_selected = all_results[[
54     'homewin_chance', 'draw_chance', 'awaywin_chance',
55     'homewin_nogoalnoled', 'draw_nogoalnoled', 'awaywin_nogoalnoled'
56 ]]
57 df_selected.columns = [
58     'clean_av_home', 'clean_av_draw', 'clean_av_away',
59     'clean_av_home_nogoalnoled', 'clean_av_draw_nogoalnoled',
60     'clean_av_away_nogoalnoled'
61 ]
62
63 dataset = dataset.reset_index(drop=True)
64 dataset = pd.concat([dataset, df_selected], axis=1)
65 dataset.to_csv(OUTPUT_PATH, index=False, encoding="utf-8-sig")
66 print(f"Processed file saved to: {OUTPUT_PATH}")

```

Explanation: Each match is processed independently by slicing the full dataset into 92-row blocks, one block per game. The `game_betting_odds()` function coordinates the simulation and probability calculation for a single game, returning six arrays of length 92: the standard live odds and the no-goal-no-red odds for each minute. These are assembled into a DataFrame and concatenated across all games.

The output columns are renamed to `clean_av_home`, `clean_av_draw`, `clean_av_away` (standard live odds) and `clean_av_home_nogoalnoled`, `clean_av_draw_nogoalnoled`, `clean_av_away_nogoalnoled` (contamination-free odds). These six columns are appended to the original dataset and saved as a single CSV file, which serves as the input for Step 4.

Conclusion of Step 3

Step 3 produces the minute-by-minute live betting odds that form the backbone of the suspense measure. By simulating 100,000 possible match futures from every minute forward, we obtain a data-driven estimate of the true win, draw, and loss probabilities at each point in time. The no-goal-no-red variant ensures that the baseline used in the suspense calculation is not contaminated by events that have just occurred in the current minute, addressing a subtle but important methodological concern. Both sets of odds are carried forward into Step 4.

Step 4: Calculating Suspense

Suspense at minute t captures how much the outcome probabilities *would* change if a goal were scored in minute $t + 1$, weighted by the probability that such a goal actually occurs. This is a forward-looking measure: it reflects the tension a viewer feels *now* about what *might* happen *next*. The current minute's live odds serve as the emotional baseline, while the next minute's hypothetical probabilities and scoring rates determine the potential magnitude of change.

(a) Importing libraries, setting seed, and loading data

```
1 import pandas as pd
2 import numpy as np
3 import os
4
5 np.random.seed(482931)
6
7 INPUT_FILE = 'live_betting_odds_output.csv'
8 OUTPUT_PATH = 'suspense_output.csv'
9
10 dataset = pd.read_csv(INPUT_FILE)
```

Explanation: The same fixed random seed is used here as in Step 3 to maintain consistency and reproducibility across the full pipeline. The input file is the output from Step 3, which contains the minute-by-minute dataset with the live betting odds (`clean_av_home`, `clean_av_draw`, `clean_av_away`) appended.

(b) Simulations and variable extraction

The `score_separator()`, `all_variables()`, and `simulations()` functions in this step are identical to those in Step 3, with one important difference in `all_variables()`: the baseline odds are now read from `clean_av_home`, `clean_av_draw`, and `clean_av_away` (the live betting odds computed in Step 3) rather than from the pre-match closing odds.

```
1 def all_variables(dataset):
2     score = dataset['score'].dropna()
3     red = dataset['red'].dropna().tolist()
4     homered, awayred = score_separator(red)
5     homescored, awayscored = score_separator(score)
6     homescoring = dataset['homescoring'].dropna().values
7     awayscoring = dataset['awayscoring'].dropna().values
8
9     # Live betting odds used as the baseline (not pre-match odds)
10    home_win = dataset['clean_av_home'].dropna().tolist()
11    all_draw = dataset['clean_av_draw'].dropna().tolist()
12    away_win = dataset['clean_av_away'].dropna().tolist()
13
14    return [
15        homered, awayred,
16        homescored, awayscored,
17        homescoring, awayscoring,
18        home_win, all_draw, away_win
19    ]
```

Explanation: Using live betting odds as the baseline is essential for the suspense measure. Pre-match odds reflect the bookmaker's assessment before any information about the match is available. Live odds, by contrast, reflect the actual match state at each minute — the current score, time remaining, and any red cards. Suspense is conceptually about how much the *current* situation could change, so the baseline must reflect how things *currently* stand.

(c) Calculating hypothetical win probabilities: home goal scenario

```

1 def calculate_chances_homescore(home_binomials, away_binomials,
2                                 homescored, awayscored):
3     num_minutes, _ = home_binomials.shape
4     homewin_chance = np.zeros(num_minutes)
5     draw_chance = np.zeros(num_minutes)
6     awaywin_chance = np.zeros(num_minutes)
7
8     for i in range(num_minutes):
9         # Add +1 to home team's score to simulate a hypothetical home goal
10        home_goals = home_binomials[i:, :].sum(axis=0) + homescored[i] + 1
11        away_goals = away_binomials[i:, :].sum(axis=0) + awayscored[i]
12
13        diff = home_goals - away_goals
14        homewin_chance[i] = np.mean(diff > 0)
15        draw_chance[i] = np.mean(diff == 0)
16        awaywin_chance[i] = np.mean(diff < 0)
17
18    return homewin_chance, draw_chance, awaywin_chance

```

Explanation: This function calculates what the outcome probabilities *would be* if the home team scored a goal at minute i . By adding +1 to the home team's current cumulative score before summing the remaining simulated goals, we simulate the downstream consequences of a hypothetical home goal. The function returns three arrays of length 92: the probability of a home win, draw, and away win *given that the home team scores at minute i* . These hypothetical probabilities are compared against the current live odds in the suspense formula to measure how impactful such a goal would be.

(d) Calculating hypothetical win probabilities: away goal scenario

```

1 def calculate_chances_awayscore(home_binomials, away_binomials,
2                                 homescored, awayscored):
3     num_minutes, _ = home_binomials.shape
4     homewin_achance = np.zeros(num_minutes)
5     draw_achance = np.zeros(num_minutes)
6     awaywin_achance = np.zeros(num_minutes)
7
8     for i in range(num_minutes):
9         # Add +1 to away team's score to simulate a hypothetical away goal
10        home_goals = home_binomials[i:, :].sum(axis=0) + homescored[i]
11        away_goals = away_binomials[i:, :].sum(axis=0) + awayscored[i] + 1
12
13        diff = home_goals - away_goals
14        homewin_achance[i] = np.mean(diff > 0)
15        draw_achance[i] = np.mean(diff == 0)
16        awaywin_achance[i] = np.mean(diff < 0)
17
18    return homewin_achance, draw_achance, awaywin_achance

```

Explanation: This function mirrors the home goal scenario but introduces a hypothetical away goal instead. By adding +1 to the away team's cumulative score, we compute how the outcome probabilities would shift if the away team were to score at minute i . Together with `calculate_chances_homescore()`, this provides all the ingredients needed for the suspense formula: hypothetical probabilities for both possible scoring events in the next minute.

(e) Forward-looking shift and the suspense formula

```

1 def game_suspense(game_data):
2     (homered, awayred, homescored, awayscored,
3      homescoring, awayscoring,
4      home_win, all_draw, away_win) = game_data
5
6     home_win = np.array(home_win)
7     all_draw = np.array(all_draw)

```



```

8 away_win = np.array(away_win)
9 homescoring = np.array(homescoring)
10 awayscoring = np.array(awayscoring)
11
12 home_binomials, away_binomials = simulations(
13     homered, awayred, homescoring, awayscoring
14 )
15
16 homewin_chance, draw_chance, awaywin_chance = \
17     calculate_chances_homescore(
18         home_binomials, away_binomials, homescored, awayscored
19     )
20
21 homewin_achance, draw_achance, awaywin_achance = \
22     calculate_chances_awayscore(
23         home_binomials, away_binomials, homescored, awayscored
24     )
25
26 # Roll hypothetical probs and scoring rates forward by 1 minute
27 # Suspense at minute t uses:
28 # - current minute's live odds as the baseline (how I feel NOW)
29 # - next minute's hypothetical probs and scoring rates (what could happen NEXT)
30 homewin_chance_next = np.roll(homewin_chance, -1); homewin_chance_next[-1] = 0
31 draw_chance_next = np.roll(draw_chance, -1); draw_chance_next[-1] = 0
32 awaywin_chance_next = np.roll(awaywin_chance, -1); awaywin_chance_next[-1] = 0
33 homewin_achance_next = np.roll(homewin_achance, -1); homewin_achance_next[-1] = 0
34 draw_achance_next = np.roll(draw_achance, -1); draw_achance_next[-1] = 0
35 awaywin_achance_next = np.roll(awaywin_achance, -1); awaywin_achance_next[-1] = 0
36 homescoring_next = np.roll(homescoring, -1); homescoring_next[-1] = 0
37 awayscoring_next = np.roll(awayscoring, -1); awayscoring_next[-1] = 0
38
39 suspense = np.sqrt(
40     homescoring_next * (homewin_chance_next - home_win) ** 2 +
41     homescoring_next * (draw_chance_next - all_draw) ** 2 +
42     homescoring_next * (awaywin_chance_next - away_win) ** 2 +
43     awayscoring_next * (homewin_achance_next - home_win) ** 2 +
44     awayscoring_next * (draw_achance_next - all_draw) ** 2 +
45     awayscoring_next * (awaywin_achance_next - away_win) ** 2
46 )
47
48 return (
49     suspense,
50     homewin_chance, draw_chance, awaywin_chance,
51     homewin_achance, draw_achance, awaywin_achance
52 )

```

Explanation: This is the core of the suspense calculation. The formula computes, at each minute t , how much the match outcome probabilities would change if either team scored in minute $t + 1$, weighted by the probability of scoring in that next minute.

The forward-looking shift is implemented using `np.roll(..., -1)`, which moves each array element one position earlier. This means that the hypothetical probabilities and scoring rates assigned to minute t are those that were calculated *for* minute $t + 1$. The last minute is zeroed out since there is no next minute to look forward to.

Conceptually, suspense at minute t answers the question: “how much would I care if a goal happened next minute, given how things currently stand?” The current live odds (`home_win`, `all_draw`, `away_win`) represent how the viewer currently feels, while the next minute’s hypothetical probabilities represent what would change. The scoring rates from the next minute weight each term by the actual probability of a goal occurring. The full formula is:

$$\text{SUS}_t = \sqrt{\sum_{k \in \{H,D,A\}} \lambda_{t+1}^{\text{home}} \left(p_{k,t+1}^{\text{home-goal}} - p_{k,t}^{\text{live}} \right)^2 + \sum_{k \in \{H,D,A\}} \lambda_{t+1}^{\text{away}} \left(p_{k,t+1}^{\text{away-goal}} - p_{k,t}^{\text{live}} \right)^2} \quad (1)$$

where $\lambda_{t+1}^{\text{home}}$ and $\lambda_{t+1}^{\text{away}}$ are the per-minute scoring rates at minute $t + 1$, $p_{k,t}^{\text{live}}$ is the current live probability

of outcome k at minute t , and $p_{k,t+1}^{home-goal}$ (respectively $p_{k,t+1}^{away-goal}$) is the probability of outcome k if the home (respectively away) team scores at minute $t + 1$.

(f) Processing all games and saving output

```

1 def process_games(dataset):
2     num_minutes = 92
3     num_games = len(dataset) // num_minutes
4     results = []
5
6     for game_idx in range(num_games):
7         start = game_idx * num_minutes
8         end = start + num_minutes
9         game_data = [var[start:end] for var in all_variables(dataset)]
10
11         (suspense,
12          homewin_chance, draw_chance, awaywin_chance,
13          homewin_achance, draw_achance, awaywin_achance
14          ) = game_suspense(game_data)
15
16         game_results = pd.DataFrame({
17             'suspense':      suspense,
18             'homewin_chance': homewin_chance,
19             'draw_chance':   draw_chance,
20             'awaywin_chance': awaywin_chance,
21             'homewin_achance': homewin_achance,
22             'draw_achance':  draw_achance,
23             'awaywin_achance': awaywin_achance
24         })
25         results.append(game_results)
26         print(f"Game_{game_idx+1}_finished")
27
28     return pd.concat(results, ignore_index=True)
29
30 all_results = process_games(dataset)
31
32 dataset = dataset.reset_index(drop=True)
33 dataset = pd.concat([dataset, all_results], axis=1)
34 dataset.to_csv(OUTPUT_PATH, index=False, encoding="utf-8-sig")
35 print(f"Suspense_file_saved_to:{OUTPUT_PATH}")

```

Explanation: Each game is processed independently by slicing the dataset into 92-row blocks. The suspense value and all hypothetical probability arrays are returned for each game and assembled into a DataFrame. All input columns from the live betting odds file are preserved, with the suspense and hypothetical probability columns appended. The final output file serves as the input for Step 5, where surprise and shock are calculated.

Conclusion of Steps 3 and 4

Together, Steps 3 and 4 produce the minute-by-minute suspense measure for every match. Step 3 establishes the live betting odds that serve as the emotional baseline, or how fans currently perceive the match outcome. Step 4 builds on this by introducing hypothetical scoring scenarios and computing how dramatically the situation would change if either team scored in the next minute, weighted by the probability of that happening. The result is a rich, continuous measure of match tension that varies dynamically across all 92 minutes of each game.

Step 5: Calculating Surprise and Shock

While suspense is forward-looking, measuring anticipated tension about what might happen next, surprise and shock are backward-looking measures that quantify the emotional impact of what has already happened. Surprise captures the immediate change in outcome probabilities from one minute to the next, reflecting the unexpectedness of recent match events. In this paper, surprise is the primary backward-looking measure used in the analysis. Shock captures the total deviation from pre-match expectations at any given point in the match and is not used as a standalone measure in this paper. Instead, shock serves a specific technical role: in the rare case where a goal or red card occurs in the very first minute of a match, the standard surprise formula cannot be applied since there is no previous minute to compare against. In such cases, the shock value, which compares the current live odds against the pre-match closing odds, is substituted in place of surprise for that minute, ensuring that the emotional impact of these early events is still captured rather than being assigned a value of zero

(a) Importing libraries and loading data

```
1 import pandas as pd
2 import numpy as np
3 import os
4
5 INPUT_PATH = 'suspense_output.csv'
6 OUTPUT_PATH = 'surprise_shock_output.csv'
7
8 df = pd.read_csv(INPUT_PATH)
9 df.sort_values(by=['odds_matchdate_id', 'minute'], inplace=True)
```

Explanation: The input file for this step is the suspense output from Step 4, which already contains the live betting odds (`clean_av_home`, `clean_av_draw`, `clean_av_away`) and pre-match odds (`prehome`, `predraw`, `preaway`) alongside the suspense values. Sorting by match ID and minute ensures that the minute-to-minute differences are computed correctly within each game rather than across game boundaries.

(b) Calculating Surprise

```
1 df['homeprobminus1'] = df.groupby('odds_matchdate_id')['clean_av_home'].diff()
2 df['drawprobminus1'] = df.groupby('odds_matchdate_id')['clean_av_draw'].diff()
3 df['awayprobminus1'] = df.groupby('odds_matchdate_id')['clean_av_away'].diff()
4
5 df['home_prob_sq'] = df['homeprobminus1'] ** 2
6 df['draw_prob_sq'] = df['drawprobminus1'] ** 2
7 df['away_prob_sq'] = df['awayprobminus1'] ** 2
8
9 df['surprise'] = np.sqrt(
10     df['home_prob_sq'] +
11     df['draw_prob_sq'] +
12     df['away_prob_sq']
13 )
14 df['surprise'].fillna(0, inplace=True)
15
16 df.drop(columns=[
17     'homeprobminus1', 'drawprobminus1', 'awayprobminus1',
18     'home_prob_sq', 'draw_prob_sq', 'away_prob_sq'
19 ], inplace=True)
```

Explanation: Surprise at minute t is defined as the Euclidean distance between the outcome probability vector at minute t and the probability vector at minute $t - 1$:

$$\text{SUR}_t = \sqrt{(p_{H,t}^{\text{live}} - p_{H,t-1}^{\text{live}})^2 + (p_{D,t}^{\text{live}} - p_{D,t-1}^{\text{live}})^2 + (p_{A,t}^{\text{live}} - p_{A,t-1}^{\text{live}})^2} \quad (2)$$

The `groupby().diff()` operation ensures that differences are only computed within each match, preventing any spillover between games. The first minute of each game has no previous minute and is therefore assigned a

surprise value of 0 by default (via `fillna(0)`), unless a goal or red card occurred in minute 1, in which case it is replaced with the shock value (see subsection d). Intermediate columns are dropped to keep the output clean.

(c) Calculating Shock

```

1 df['homeprobminus0'] = df['clean_av_home'] - df['prehome']
2 df['drawprobminus0'] = df['clean_av_draw'] - df['predraw']
3 df['awayprobminus0'] = df['clean_av_away'] - df['preaway']
4
5 df['home_prob_sq'] = df['homeprobminus0'] ** 2
6 df['draw_prob_sq'] = df['drawprobminus0'] ** 2
7 df['away_prob_sq'] = df['awayprobminus0'] ** 2
8
9 df['shock'] = np.sqrt(
10     df['home_prob_sq'] +
11     df['draw_prob_sq'] +
12     df['away_prob_sq']
13 )
14
15 df.drop(columns=[
16     'homeprobminus0', 'drawprobminus0', 'awayprobminus0',
17     'home_prob_sq', 'draw_prob_sq', 'away_prob_sq'
18 ], inplace=True)

```

Explanation: Shock at minute t measures the total deviation of the current live probabilities from the pre-match closing odds:

$$\text{SHO}_t = \sqrt{\left(p_{H,t}^{\text{live}} - p_H^{\text{pre}}\right)^2 + \left(p_{D,t}^{\text{live}} - p_D^{\text{pre}}\right)^2 + \left(p_{A,t}^{\text{live}} - p_A^{\text{pre}}\right)^2} \quad (3)$$

Unlike surprise, which measures change from the previous minute, shock is an absolute measure relative to the pre-match baseline. A large shock value indicates that the match has evolved in a way that was very unexpected relative to pre-match expectations — for example, a heavy favourite going behind early. Intermediate columns are dropped after the calculation.

(d) Replacing Surprise with Shock in Minute 1

```

1 mask_min1_goal_red = (
2     (df['minute'] == 1) &
3     ((df['score'] != "0,0") | (df['red'] != "0,0"))
4 )
5
6 df.loc[mask_min1_goal_red, 'surprise'] = df.loc[mask_min1_goal_red, 'shock']
7 df.drop(columns=['shock'], inplace=True)

```

Explanation: In the first minute of a match, surprise is undefined in the standard sense because there is no previous minute to compare against. Normally this is handled by assigning a value of 0. However, if a goal or red card occurs in minute 1, a value of 0 would fail to capture the genuine emotional impact of that event. In such cases, the shock value — which compares the current live odds against the pre-match odds — serves as a meaningful substitute, capturing the full magnitude of the deviation from expectations caused by the early event. The shock column is dropped after this adjustment as it is no longer needed.

(e) Saving output

```

1 df.to_csv(OUTPUT_PATH, index=False, encoding="utf-8-sig")
2 print(f"Surprise file saved to: {OUTPUT_PATH}")

```

Explanation: The final dataset, containing all original columns alongside the computed surprise values, is saved to a CSV file. This file represents the complete output of the pipeline and is ready for further statistical analysis, aggregation, and visualization.

Conclusion of Step 5

Step 5 completes the pipeline by adding two complementary backward-looking emotion measures to the dataset. Surprise captures the immediate, minute-to-minute unexpectedness of match events, reflecting the emotional jolt experienced when something changes suddenly. Shock captures the cumulative deviation from pre-match expectations at any point in the game, reflecting how far the match has strayed from what was anticipated before kick-off. Together with the suspense measure from Step 4, these three metrics provide a comprehensive minute-by-minute characterization of the emotional dynamics of a soccer match.

Conclusion

This document has described the complete pipeline for calculating suspense, surprise, and shock in soccer matches. The pipeline proceeds through five interconnected steps, each building directly on the output of the previous one.

Step 1 fits Poisson-based scoreline models to bookmaker over/under odds, yielding match-specific home and away scoring rates. Step 2 distributes these rates across the 92 minutes of a match using an empirical goal distribution, producing per-minute scoring probabilities. Step 3 uses these probabilities to simulate 100,000 possible match futures from every minute forward, generating live betting odds that reflect the true match state, including score, time remaining, and red cards. A contamination-free variant of these odds is also produced to serve as a clean baseline for the suspense calculation. Step 4 introduces hypothetical scoring scenarios to compute the suspense measure: at each minute, suspense captures how much the outcome probabilities would change if either team scored in the next minute, weighted by the actual probability of that occurring. Step 5 calculates surprise and shock, two backward-looking measures that quantify the emotional impact of what has already happened, surprise measuring minute-to-minute changes and shock measuring total deviation from pre-match expectations.

The full pipeline is implemented in Python using standard scientific libraries (`numpy`, `pandas`, `scipy`) and results are fully reproducible via a fixed random seed of 482931. All scripts, along with the preprocessing steps not covered in this document, are available in the accompanying GitHub repository under an MIT license, making the code freely available for academic and personal use. Researchers making use of this code or methodology in their own work are kindly asked to cite the original paper. The full replication package including all preprocessing scripts is available at: [GitHub URL to be added upon publication].